Microservices Interview Questions & Answers

Table of Contents

1. [Redis](#redis-interview-questions)
2. [gRPC](#grpc-interview-questions)
3. [REST vs gRPC](#rest-vs-grpc-interview-questions)
4. [Background Services](#background-services-interview-questions)
5. [Polly & Resilience](#polly--resilience-interview-questions)
6. [Elasticsearch](#elasticsearch-interview-questions)
7. [RabbitMQ](#rabbitmq-interview-questions)
8. [Saga Pattern](#saga-pattern-interview-questions)
9. [CQRS Pattern](#cqrs-pattern-interview-questions)
10. [Mediator Pattern](#mediator-pattern-interview-questions)
11. [API Versioning](#api-versioning-interview-questions)
12. [Authorization](#authorization-interview-questions)
13. [MSAL](#msal-interview-questions)
14. [Seed Data](#seed-data-interview-questions)
15. [Factory Pattern](#factory-pattern-interview-questions)
16. [Abstract Factory](#abstract-factory-interview-questions)

Redis Interview Questions

Q1: Why did you choose Redis for your basket service instead of a traditional database?

Answer: I chose Redis for the basket service because shopping carts are temporary, session-based data that requires fast access and frequent updates. Redis provides:

1. **In-memory storage** - Extremely fast read/write operations (sub-millisecond latency)
2. **Session persistence** - User baskets need to persist during the shopping session
3. **Simple key-value structure** - Perfect for basket storage where username is the key

In my implementation, I used `IDistributedCache` which abstracts Redis operations:

```csharp
// Basket.Infrastructure/Repositories/BasketRepository.cs
```

```csharp
public async Task<ShoppingCart> GetBasket(string userName)
{
    var basket = await _redisCache.GetStringAsync(userName);
    if(string.IsNullOrEmpty(basket))
    {
        return null;
    }
    return JsonConvert.DeserializeObject<ShoppingCart>(basket);
}
```

This approach provides better performance than querying a SQL database for every basket operation.

Q2: How do you handle Redis failures in your microservices architecture?

Answer: In my current implementation, I don't have explicit Redis failure handling, but in production I would implement:

1. **Polly retry policy** - For transient Redis failures
2. **Fallback mechanism** - Store baskets in SQL Server as backup
3. **Circuit breaker** - Prevent cascading failures
4. **Health checks** - Monitor Redis availability

Here's how I would implement it:

```csharp
// Enhanced implementation with resilience
var redisPolicy = Policy
    .Handle<RedisConnectionException>()
    .Or<RedisTimeoutException>()
    .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2,
retryAttempt)))
    .FallbackAsync(async () =>
    {
        // Fallback to SQL Server
        return await _sqlBasketRepository.GetBasket(userName);
    });
```

Q3: What Redis data structures would you consider for different e-commerce scenarios?

Answer: For e-commerce, I would use:

1. **String** - For basket storage (current implementation)
2. **Hash** - For user profiles with multiple fields
3. **Sorted Set** - For product rankings and leaderboards
4. **List** - For recent viewed products
5. **Set** - For product tags and categories

Example for product rankings:

```csharp
// Using Sorted Set for product popularity
await _redisDb.SortedSetAddAsync("product:rankings", productId, score);
```

---

gRPC Interview Questions

Q1: Why did you implement gRPC for inter-service communication instead of REST?

Answer: I chose gRPC for inter-service communication because:

1. **Performance** - Binary serialization with Protocol Buffers is 5-10x faster than JSON
2. **Type safety** - Compile-time contract validation prevents runtime errors
3. **Code generation** - Auto-generated client/server stubs reduce boilerplate
4. **Streaming** - Support for bidirectional streaming (though not currently used)

In my project, the Basket service calls the Discount service via gRPC:

```csharp
// Basket.Application/GrpcService/DiscountGrpcService.cs
public async Task<CouponModel> GetDiscount(string productName)
{
    var discountRequest = new GetDiscountRequest { ProductName = productName };
    return await _discountProtoServiceClient.GetDiscountAsync(discountRequest);
}
```

This is much more efficient than making HTTP REST calls with JSON serialization overhead.

Q2: How do you handle gRPC service discovery and load balancing?

Answer: In my current Docker setup, I use hardcoded service names, but in production I would implement:

1. **Kubernetes service discovery** - Using Kubernetes DNS
2. **Envoy proxy** - For advanced load balancing and retries
3. **Consul/etcd** - For service registration and discovery

My current configuration:

```csharp
// Program.cs (Basket.API)
builder.Services.AddGrpcClient<DiscountProtoService.DiscountProtoServiceClient>(o
=>
{
    o.Address = new Uri(builder.Configuration["GrpcSettings:DiscountUrl"]);
});
```

In production, this would be:
```csharp
o.Address = new Uri("http://discount-service:8080");
```

Q3: What's the difference between unary and streaming gRPC calls?

Answer:

Unary calls (what I currently use):
- One request, one response
- Simple request/response pattern
- Good for single operations like getting a discount

```csharp
// My current unary implementation
public override async Task<CouponModel> GetDiscount(GetDiscountRequest request,
ServerCallContext context)
{
```

```
    var query = new GetDiscountQuery(request.ProductName);
    var result = await _mediator.Send(query);
    return result;
}
```

Streaming calls:
- **Client streaming** - Client sends multiple messages, server responds once
- **Server streaming** - Client sends one message, server streams multiple responses
- **Bidirectional streaming** - Both client and server can send multiple messages

For e-commerce, I could use server streaming for:
- Real-time inventory updates
- Order status notifications
- Product recommendations

REST vs gRPC Interview Questions

Q1: How do you decide when to use REST vs gRPC in your microservices architecture?

Answer: I follow this decision matrix:

Use REST for:
- Frontend communication (browser compatibility)
- Public APIs (external consumers)
- Simple CRUD operations
- When tooling and debugging are priorities

Use gRPC for:
- Inter-service communication (internal)
- High-performance requirements
- Type safety is critical
- Complex object graphs

In my project, I use both:
```csharp
// Frontend → Basket API (REST)
[HttpGet]
```

```
public async Task<ActionResult<ShoppingCartResponse>> GetBasket(string userName)

// Basket API → Discount API (gRPC)
var discount = await _discountGrpcService.GetDiscount(item.ProductName);
```

This hybrid approach gives me the best of both worlds.

Q2: What are the performance implications of your hybrid approach?

Answer: The hybrid approach provides optimal performance:

1. **Frontend communication** - REST is sufficient as network latency dominates
2. **Inter-service** - gRPC reduces serialization overhead and network calls
3. **Protocol efficiency** - HTTP/2 multiplexing vs HTTP/1.1

Performance comparison in my context:
```
REST (Frontend → Basket): ~50ms (network dominated)
gRPC (Basket → Discount): ~5ms (binary serialization)
```

The gRPC calls are an order of magnitude faster due to:
- Binary Protocol Buffers vs JSON
- HTTP/2 multiplexing
- Generated code eliminates reflection

Background Services Interview Questions

Q1: How would you implement background job processing in your e-commerce system?

Answer: I would implement Hangfire for several e-commerce scenarios:

1. **Order confirmation emails** - Fire-and-forget jobs
2. **Daily sales reports** - Recurring jobs
3. **Inventory reconciliation** - Delayed jobs
4. **Payment processing** - Continuations

Here's how I'd implement it:

```csharp
// OrderService with background jobs
public async Task<IActionResult> CreateOrder(Order order)
{
    // Create order
    await _orderService.CreateOrder(order);

    // Send confirmation email (fire-and-forget)
    BackgroundJob.Enqueue<IEmailService>(x => x.SendOrderConfirmation(order.Id));

    // Process payment asynchronously
    var paymentJobId = BackgroundJob.Enqueue<IPaymentService>(x =>
x.ProcessPayment(order.Id));

    // Update order status after payment (continuation)
    BackgroundJob.ContinueJobWith(paymentJobId,<IOrderService>(x =>
x.UpdateOrderStatus(order.Id, OrderStatus.Processing)));

    return Ok(order);
}

// Daily report generation
public class ReportService
{
    [AutomaticRetry(Attempts = 3)]
    public void GenerateDailySalesReport()
    {
        // Generate and email daily sales report
        var report = _reportService.GenerateSalesReport(DateTime.Today);
        _emailService.SendReport(report);
    }
}

// Configure in Program.cs
builder.Services.AddHangfire(config =>
{
    config.UseSimpleAssemblyNameTypeSerializer();
    config.UseRecommendedSerializerSettings();

config.UseSqlServerStorage(builder.Configuration.GetConnectionString("DefaultConn
ection"));
});
```

```
builder.Services.AddHangfireServer();
app.UseHangfireDashboard();
```

Q2: How do you ensure idempotency in background jobs?

Answer: For idempotency, I implement:

1. **Idempotency keys** - Track processed jobs
2. **Deduplication** - Check if job already processed
3. **Database constraints** - Prevent duplicate inserts

```csharp
public class PaymentService
{
    public async Task ProcessPayment(string orderId)
    {
        // Check if already processed
        if (await _paymentRepository.IsProcessed(orderId))
        {
            _logger.LogInformation($"Payment for order {orderId} already
processed");
            return;
        }

        // Process payment
        await _paymentProcessor.Charge(order);

        // Mark as processed
        await _paymentRepository.MarkAsProcessed(orderId);
    }
}
```

Polly & Resilience Interview Questions

Q1: How would you add resilience to your gRPC calls using Polly?

Answer: I would implement multiple resilience patterns for gRPC calls:

```csharp
// Enhanced gRPC client with Polly
builder.Services.AddGrpcClient<DiscountProtoService.DiscountProtoServiceClient>(o
=>
{
    o.Address = new Uri(builder.Configuration["GrpcSettings:DiscountUrl"]);
})
.ConfigureChannel(o =>
{
    o.HttpHandler = new PollyHandler(new ResiliencePipeline<HttpResponseMessage>
    {
        // Retry for transient failures
        new RetryStrategy<HttpResponseMessage>
        {
            ShouldRetry = args => args.Outcome.Exception is RpcException,
            Delay = TimeSpan.FromSeconds(1),
            MaxRetryAttempts = 3
        },
        // Circuit breaker for consecutive failures
        new CircuitBreakerStrategy<HttpResponseMessage>
        {
            FailureThreshold = 5,
            SamplingDuration = TimeSpan.FromSeconds(30),
            MinimumThroughput = 1
        },
        // Timeout for slow responses
        new TimeoutStrategy<HttpResponseMessage>
        {
            Timeout = TimeSpan.FromSeconds(5)
        }
    });
});
```

Q2: What resilience patterns would you implement for your basket checkout flow?

Answer: For the basket checkout flow, I'd implement:

1. **Retry** - For transient database failures
2. **Circuit breaker** - For Discount service failures
3. **Fallback** - For RabbitMQ unavailability

4. **Timeout** - For external service calls

```csharp
public class BasketService
{
    private readonly ResiliencePipeline _discountPipeline;
    private readonly ResiliencePipeline _messagingPipeline;

    public async Task<IActionResult> Checkout(BasketCheckout checkout)
    {
        try
        {
            // Get basket with retry
            var basket = await _retryPipeline.ExecuteAsync(async () =>
                await _basketRepository.GetBasket(checkout.UserName));

            // Get discount with circuit breaker
            var discount = await _discountPipeline.ExecuteAsync(async () =>
                await _discountService.GetDiscount(checkout.ProductName));

            // Publish event with fallback
            await _messagingPipeline.ExecuteAsync(async () =>
                await _eventBus.Publish(new BasketCheckoutEvent(checkout)));

            return Accepted();
        }
        catch (BrokenCircuitException)
        {
            // Fallback when circuit is open
            return StatusCode(503, "Service temporarily unavailable");
        }
    }
}
```

Elasticsearch Interview Questions

Q1: Why did you choose Elasticsearch for logging in your microservices?

Answer: I chose Elasticsearch for several reasons:

1. **Full-text search** - Powerful search capabilities across log messages
2. **Scalability** - Horizontal scaling across multiple nodes
3. **Real-time** - Near real-time indexing and search
4. **Kibana integration** - Powerful visualization and dashboarding

In my implementation:

```csharp
// Infrastructure/Common.Logging/Logging.cs
loggerConfiguration.WriteTo.Elasticsearch(
    new ElasticsearchSinkOptions(new Uri(elasticUrl))
    {
        AutoRegisterTemplate = true,
        AutoRegisterTemplateVersion = AutoRegisterTemplateVersion.ESv8,
        IndexFormat = $"{serviceName}-Logs-{0:yyyy.MM.dd}",
        MinimumLogEventLevel = LogEventLevel.Debug
    });
```

This creates daily indices per service, making it easy to:
- Search logs by service
- Filter by date range
- Analyze log patterns across services

Q2: How do you correlate logs across multiple microservices?

Answer: I implement distributed tracing with correlation IDs:

```csharp
// In BasketCheckoutConsumer
public async Task Consume(ConsumeContext<BasketCheckoutEvent> context)
{
    using var scope = _logger.BeginScope("Consume Basket Checkout Event For
{correlationId}", context.Message.CorrelationId);

    var cmd = _mapper.Map<CheckoutOrderCommand>(context.Message);
    var result = await _mediator.Send(cmd);

    _logger.LogInformation("Basket Checkout Event completed !!");
}
```

The correlation ID flows through:
1. Frontend request → Basket API
2. Basket API → RabbitMQ event
3. RabbitMQ → Ordering API consumer
4. All logs include the same correlation ID

This allows me to trace a single request across all services in Kibana.

RabbitMQ Interview Questions

Q1: How did you implement event versioning in your RabbitMQ setup?

Answer: I implemented event versioning using separate queues and event classes:

```csharp
// V1 Event
public class BasketCheckoutEvent : BaseIntegrationEvent
{
    public string? UserName { get; set; }
    public decimal? TotalPrice { get; set; }
    public string? FirstName { get; set; }
    public string? LastName { get; set; }
    public string? EmailAddress { get; set; }
    // ... more fields
}

// V2 Event (simplified)
public class BasketCheckoutEventV2 : BaseIntegrationEvent
{
    public string? UserName { get; set; }
    public decimal? TotalPrice { get; set; }
}
```

And separate consumers:

```csharp
// V1 Consumer
```

```csharp
public class BasketOrderingConsumer : IConsumer<BasketCheckoutEvent>
{
    public async Task Consume(ConsumeContext<BasketCheckoutEvent> context)
    {
        var cmd = _mapper.Map<CheckoutOrderCommand>(context.Message);
        var result = await _mediator.Send(cmd);
    }
}

// V2 Consumer
public class BasketOrderingConsumerV2 : IConsumer<BasketCheckoutEventV2>
{
    public async Task Consume(ConsumeContext<BasketCheckoutEventV2> context)
    {
        var cmd = _mapper.Map<CheckoutOrderCommandV2>(context.Message);
        var result = await _mediator.Send(cmd);
    }
}
```

This allows:
- Backward compatibility (V1 consumers continue working)
- Gradual migration (new services use V2)
- A/B testing (run both versions simultaneously)

Q2: How do you handle message ordering and exactly-once delivery?

Answer: For message ordering, I use:

1. **Single consumer per queue** - Guarantees FIFO order
2. **Message sequence numbers** - For ordering across partitions
3. **Idempotent consumers** - Handle duplicate messages

```csharp
public class OrderingConsumer
{
    private readonly HashSet<string> _processedMessages = new();

    public async Task Consume(ConsumeContext<BasketCheckoutEvent> context)
    {
        var messageId = context.MessageId.ToString();

        // Check if already processed (idempotency)
```

```csharp
        if (_processedMessages.Contains(messageId))
        {
            _logger.LogWarning($"Message {messageId} already processed");
            return;
        }

        // Process message
        await _orderService.CreateOrder(context.Message);

        // Mark as processed
        _processedMessages.Add(messageId);

        // Cleanup old messages (optional)
        if (_processedMessages.Count > 10000)
        {
            _processedMessages.RemoveWhere(id => id != messageId);
        }
    }
}
```

For exactly-once delivery, I implement:
- Database transactions
- Message deduplication
- Acknowledgment after successful processing

Saga Pattern Interview Questions

Q1: How would you implement the Saga pattern for order processing in your system?

Answer: I would implement a choreography-based Saga for order processing:

```csharp
// Order Service - Orchestrates the saga
public class OrderService
{
    public async Task CreateOrder(Order order)
    {
        // Step 1: Create order (PENDING)
```

```
        order.Status = OrderStatus.Pending;
        await _orderRepository.CreateOrder(order);

        // Publish OrderCreated event
        await _eventBus.Publish(new OrderCreatedEvent {
            OrderId = order.Id,
            TotalAmount = order.TotalAmount
        });
    }
}

// Payment Service - Handles payment
public class PaymentConsumer : IConsumer<OrderCreatedEvent>
{
    public async Task Consume(ConsumeContext<OrderCreatedEvent> context)
    {
        try
        {
            // Process payment
            await _paymentService.ProcessPayment(context.Message.OrderId,
context.Message.TotalAmount);

            // Publish PaymentCompleted event
            await _eventBus.Publish(new PaymentCompletedEvent {
                OrderId = context.Message.OrderId
            });
        }
        catch (Exception ex)
        {
            // Publish PaymentFailed event (triggers compensation)
            await _eventBus.Publish(new PaymentFailedEvent {
                OrderId = context.Message.OrderId,
                Reason = ex.Message
            });
        }
    }
}

// Inventory Service - Reserves stock
public class InventoryConsumer : IConsumer<PaymentCompletedEvent>
{
    public async Task Consume(ConsumeContext<PaymentCompletedEvent> context)
    {
```

```
        try
        {
            // Reserve inventory
            await _inventoryService.ReserveStock(context.Message.OrderId);

            // Publish InventoryReserved event
            await _eventBus.Publish(new InventoryReservedEvent {
                OrderId = context.Message.OrderId
            });
        }
        catch (Exception ex)
        {
            // Publish InventoryFailed event (triggers compensation)
            await _eventBus.Publish(new InventoryFailedEvent {
                OrderId = context.Message.OrderId,
                Reason = ex.Message
            });
        }
    }
}

// Order Service - Handles compensation
public class OrderCompensationConsumer : IConsumer<PaymentFailedEvent>,
                                         IConsumer<InventoryFailedEvent>
{
    public async Task Consume(ConsumeContext<PaymentFailedEvent> context)
    {
        // Payment failed, cancel order
        await _orderService.UpdateOrderStatus(context.Message.OrderId,
OrderStatus.Cancelled);
    }

    public async Task Consume(ConsumeContext<InventoryFailedEvent> context)
    {
        // Inventory failed, refund payment and cancel order
        await _paymentService.RefundPayment(context.Message.OrderId);
        await _orderService.UpdateOrderStatus(context.Message.OrderId,
OrderStatus.Cancelled);
    }
}
```

Q2: What are the challenges of implementing Saga pattern in your current architecture?

Answer: The main challenges are:

1. **Eventual consistency** - Data is inconsistent during saga execution
2. **Complex error handling** - Multiple compensation scenarios
3. **Monitoring** - Tracking saga state across services
4. **Testing** - Complex distributed scenarios

To mitigate these, I would:
- Implement saga state persistence
- Add comprehensive logging and monitoring
- Create integration tests for all failure scenarios
- Use circuit breakers for external dependencies

CQRS Pattern Interview Questions

Q1: How does your CQRS implementation handle validation using MediatR behaviors?

Answer: I implement validation using FluentValidation and MediatR pipeline behaviors:

```csharp
// ValidationBehaviour.cs
public class ValidationBehaviour<TRequest, TResponse> :
IPipelineBehavior<TRequest, TResponse>
    where TRequest : IRequest<TResponse>
{
    public async Task<TResponse> Handle(TRequest request,
RequestHandlerDelegate<TResponse> next, CancellationToken cancellationToken)
    {
        if (_validators.Any())
        {
            var context = new ValidationContext<TRequest>(request);
            var validationResults = await Task.WhenAll(_validators.Select(v =>
v.ValidateAsync(context, cancellationToken)));
            var failure = validationResults.SelectMany(e => e.Errors).Where(f =>
f != null).ToList();
```

```
            if (failure.Count != 0)
            {
                throw new FluentValidation.ValidationException(failure);
            }
        }
        return await next();
    }
}

// CheckoutOrderCommandValidator.cs
public class CheckoutOrderCommandValidator :
AbstractValidator<CheckoutOrderCommand>
{
    public CheckoutOrderCommandValidator()
    {
        RuleFor(o => o.UserName)
            .NotEmpty().WithMessage("{UserName} is Required.")
            .MaximumLength(70).WithMessage("{UserName} must not exceed 70
characters.");
        RuleFor(o => o.TotalPrice)
            .NotEmpty().WithMessage("{TotalPrice} is Required.")
            .GreaterThan(-1).WithMessage("{TotalPrice} should not be -ve");
        RuleFor(o => o.EmailAddress)
            .NotEmpty().WithMessage("{EmailAddress} is Required.");
    }
}
```

This ensures validation runs before the command handler executes, preventing invalid data from reaching the business logic.

Q2: How do you handle read model optimization in your CQRS implementation?

Answer: In my current implementation, I use simple queries, but for optimization I would:

1. **Create read-specific models** - Denormalized for queries
2. **Implement caching** - For frequently accessed data
3. **Use materialized views** - For complex aggregations

Example of optimized read model:

```csharp
// Read model for product catalog
public class ProductReadModel
{
    public string Id { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public decimal Price { get; set; }
    public string BrandName { get; set; }  // Denormalized
    public string TypeName { get; set; }   // Denormalized
    public int StockCount { get; set; }    // Denormalized
    public decimal DiscountAmount { get; set; } // Denormalized
}

// Query handler with caching
public class GetProductsHandler : IRequestHandler<GetProductsQuery,
List<ProductReadModel>>
{
    private readonly IMemoryCache _cache;

    public async Task<List<ProductReadModel>> Handle(GetProductsQuery request,
CancellationToken cancellationToken)
    {
        var cacheKey = $"products_{request.Page}_{request.PageSize}";

        if (!_cache.TryGetValue(cacheKey, out List<ProductReadModel> products))
        {
            products = await _productReadRepository.GetProducts(request);
            _cache.Set(cacheKey, products, TimeSpan.FromMinutes(5));
        }

        return products;
    }
}
```

Mediator Pattern Interview Questions

Q1: How does MediatR help implement cross-cutting concerns in your application?

Answer: MediatR enables cross-cutting concerns through pipeline behaviors:

```csharp
// Exception handling behavior
public class UnhandledExceptionBehaviour<TRequest, TResponse> :
IPipelineBehavior<TRequest, TResponse>
    where TRequest : IRequest<TResponse>
{
    public async Task<TResponse> Handle(TRequest request,
RequestHandlerDelegate<TResponse> next, CancellationToken cancellationToken)
    {
        try
        {
            return await next();
        }
        catch (Exception ex)
        {
            var requestName = typeof(TRequest).Name;
            _logger.LogError(ex, $"Unhandled exception occurred with Request
Name: {requestName}, {request}");
            throw;
        }
    }
}

// Logging behavior
public class LoggingBehaviour<TRequest, TResponse> : IPipelineBehavior<TRequest,
TResponse>
    where TRequest : IRequest<TResponse>
{
    public async Task<TResponse> Handle(TRequest request,
RequestHandlerDelegate<TResponse> next, CancellationToken cancellationToken)
    {
        _logger.LogInformation($"Handling {typeof(TRequest).Name}");
        var response = await next();
        _logger.LogInformation($"Handled {typeof(TRequest).Name}");
        return response;
    }
}
```

These behaviors automatically apply to all commands and queries, providing:
- Consistent exception handling

- Automatic logging
- Validation
- Performance monitoring

Q2: How do you test your MediatR handlers and behaviors?

Answer: I test handlers and behaviors separately:

```csharp
// Unit test for command handler
public class CreateShoppingCartCommandHandlerTests
{
    [Fact]
    public async Task Handle_ValidCommand_ShouldCreateBasket()
    {
        // Arrange
        var basketRepository = new Mock<IBasketRepository>();
        var discountService = new Mock<IDiscountGrpcService>();
        var mapper = new Mock<IMapper>();

        var handler = new CreateShoppingCartCommandHandler(
            basketRepository.Object,
            mapper.Object,
            discountService.Object);

        var command = new CreateShoppingCartCommand("testuser", new
List<ShoppingCartItem>());

        // Act
        var result = await handler.Handle(command, CancellationToken.None);

        // Assert
        basketRepository.Verify(x => x.UpdateBasket(It.IsAny<ShoppingCart>()),
Times.Once);
    }
}

// Integration test for validation behavior
public class ValidationBehaviourTests
{
    [Fact]
    public async Task Handle_InvalidCommand_ShouldThrowValidationException()
    {
```

```
        // Arrange
        var validators = new List<IValidator<CheckoutOrderCommand>>
        {
            new CheckoutOrderCommandValidator()
        };

        var behavior = new ValidationBehaviour<CheckoutOrderCommand,
Unit>(validators);
        var command = new CheckoutOrderCommand(); // Invalid command

        // Act & Assert
        await Assert.ThrowsAsync<ValidationException>(() =>
            behavior.Handle(command, () => Task.FromResult(Unit.Value),
CancellationToken.None));
    }
}
```

API Versioning Interview Questions

Q1: How did you implement API versioning in your Basket service?

Answer: I implemented URL path versioning with separate controllers:

```csharp
// V1 Controller
[ApiVersion("1")]
public class BasketController : ApiController
{
    [HttpPost("CreateBasket")]
    public async Task<ActionResult<ShoppingCartResponse>> UpdateBasket([FromBody]
CreateShoppingCartCommand command)
    {
        var basket = await _mediator.Send(command);
        return Ok(basket);
    }
}

// V2 Controller
[ApiVersion("2")]
[Route("api/v{version:apiVersion}/[controller]")]
```

```csharp
public class BasketController : ControllerBase
{
    [HttpPost("Checkout")]
    public async Task<IActionResult> Checkout([FromBody] BasketCheckoutV2
basketCheckout)
    {
        // Simplified V2 implementation
        var eventMsg = _mapper.Map<BasketCheckoutEventV2>(basketCheckout);
        await _publishEndpoint.Publish(eventMsg);
        return Accepted();
    }
}
```

Configuration:

```csharp
builder.Services
    .AddApiVersioning(options =>
    {
        options.ReportApiVersions = true;
        options.AssumeDefaultVersionWhenUnspecified = true;
        options.DefaultApiVersion = new ApiVersion(1, 0);
    })
    .AddApiExplorer(options =>
    {
        options.GroupNameFormat = "'v'VVV";
        options.SubstituteApiVersionInUrl = true;
    });
```

Q2: How do you handle breaking changes between API versions?

Answer: I handle breaking changes by:

1. **Event versioning** - Separate V1 and V2 events
2. **Backward compatibility** - V1 remains functional
3. **Gradual migration** - Clients migrate at their own pace
4. **Deprecation notices** - Inform clients of upcoming changes

Example of event evolution:

```csharp
```

```csharp
// V1 Event (Full)
public class BasketCheckoutEvent : BaseIntegrationEvent
{
    public string? UserName { get; set; }
    public decimal? TotalPrice { get; set; }
    public string? FirstName { get; set; }
    public string? LastName { get; set; }
    public string? EmailAddress { get; set; }
    public string? AddressLine { get; set; }
    public string? Country { get; set; }
    public string? State { get; set; }
    public string? ZipCode { get; set; }
    public string? CardName { get; set; }
    public string? CardNumber { get; set; }
    public string? Expiration { get; set; }
    public string? Cvv { get; set; }
    public int? PaymentMethod { get; set; }
}

// V2 Event (Simplified - removed sensitive data)
public class BasketCheckoutEventV2 : BaseIntegrationEvent
{
    public string? UserName { get; set; }
    public decimal? TotalPrice { get; set; }
}
```

Authorization Interview Questions

Q1: How would you implement role-based authorization in your microservices?

Answer: I would implement JWT-based role authorization:

```csharp
// Program.cs configuration
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
```

```
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = "ecommerce.com",
            ValidAudience = "ecommerce.com",
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes("your-secret-key"))
        };
    });

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy => policy.RequireRole("Admin"));
    options.AddPolicy("UserOrAbove", policy => policy.RequireRole("Admin",
"Manager", "User"));
});

// Controller with authorization
[Authorize(Policy = "UserOrAbove")]
[HttpGet]
public async Task<ActionResult<ShoppingCartResponse>> GetBasket(string userName)
{
    // Check if user can access their own basket
    var currentUserName = User.Identity.Name;
    if (currentUserName != userName && !User.IsInRole("Admin"))
    {
        return Forbid();
    }

    var query = new GetBasketByUserNameQuery(userName);
    var basket = await _mediator.Send(query);
    return Ok(basket);
}
```

Q2: How do you handle authorization across microservices?

Answer: I handle cross-service authorization by:

1. **JWT token propagation** - Pass token between services
2. **Service-to-service authentication** - Client credentials flow
3. **Policy enforcement** - Each service validates permissions

```csharp
// HTTP client with token propagation
public class HttpServiceClient
{
    private readonly IHttpContextAccessor _httpContextAccessor;

    public async Task<T> GetAsync<T>(string url)
    {
        var token =
_httpContextAccessor.HttpContext?.Request?.Headers["Authorization"].ToString();

        var request = new HttpRequestMessage(HttpMethod.Get, url);
        request.Headers.Authorization = new AuthenticationHeaderValue("Bearer",
token.Replace("Bearer ", ""));

        var response = await _httpClient.SendAsync(request);
        return await response.Content.ReadFromJsonAsync<T>();
    }
}

// Service-to-service authentication
public class ServiceTokenService
{
    public async Task<string> GetServiceToken()
    {
        var app = ConfidentialClientApplicationBuilder.Create(clientId)
            .WithClientSecret(clientSecret)
            .WithAuthority(new Uri($"{instance}{tenantId}"))
            .Build();

        var result = await app.AcquireTokenForClient(scopes).ExecuteAsync();
        return result.AccessToken;
    }
}
```

MSAL Interview Questions

Q1: How would you integrate MSAL with your existing microservices?

Answer: I would integrate MSAL for Azure AD authentication:

```csharp
// Program.cs
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApi(builder.Configuration.GetSection("AzureAd"));

builder.Services.AddAuthorization();

// appsettings.json
{
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "your-tenant-id",
    "ClientId": "your-client-id",
    "Domain": "yourdomain.onmicrosoft.com"
  }
}

// Protected controller
[Authorize]
public class BasketController : ApiController
{
    [HttpGet]
    public async Task<ActionResult<ShoppingCartResponse>> GetBasket()
    {
        // Get current user from token
        var userName = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;

        var query = new GetBasketByUserNameQuery(userName);
        var basket = await _mediator.Send(query);
        return Ok(basket);
    }
}
```

Q2: How do you handle token acquisition and refresh in a SPA frontend?

Answer: For Angular frontend with MSAL:

```typescript
// msal.service.ts
@Injectable({
```

```
  providedIn: 'root'
})
export class MsalService {
  private msalInstance: PublicClientApplication;

  constructor() {
    this.msalInstance = new PublicClientApplication(msalConfig);
    this.msalInstance.initialize();
  }

  async login() {
    const loginRequest = {
      scopes: ["api://your-api/access_as_user"]
    };

    await this.msalInstance.loginPopup(loginRequest);
  }

  async getAccessToken() {
    const account = this.msalInstance.getAllAccounts()[0];
    const tokenRequest = {
      scopes: ["api://your-api/access_as_user"],
      account: account
    };

    try {
      const tokenResponse = await
this.msalInstance.acquireTokenSilent(tokenRequest);
      return tokenResponse.accessToken;
    } catch (error) {
      // Fallback to interactive login
      return await this.msalInstance.acquireTokenPopup(tokenRequest);
    }
  }
}

// HTTP interceptor
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private msalService: MsalService) {}

  intercept(request: HttpRequest<unknown>, next: HttpHandler):
Observable<HttpEvent<unknown>> {
```

```
    return from(this.handleRequest(request, next));
  }

  private async handleRequest(request: HttpRequest<unknown>, next: HttpHandler) {
    const token = await this.msalService.getAccessToken();

    const authRequest = request.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });

    return next.handle(authRequest).toPromise();
  }
}
```

Seed Data Interview Questions

Q1: How do you handle seed data in different environments (dev, test, prod)?

Answer: I implement environment-specific seeding:

```csharp
// Program.cs
if (app.Environment.IsDevelopment())
{
    await SeedDevelopmentData(app);
}
else if (app.Environment.IsProduction())
{
    await SeedProductionData(app);
}

// Development seeding (full data)
public static async Task SeedDevelopmentData(IApplicationBuilder app)
{
    using var scope = app.ApplicationServices.CreateScope();
    var catalogContext =
scope.ServiceProvider.GetRequiredService<CatalogContext>();
```

```
    // Seed full product catalog
    await CatalogContextSeed.SeedData(catalogContext.Products);
    await BrandContextSeed.SeedData(catalogContext.Brands);
    await TypeContextSeed.SeedData(catalogContext.Types);
}

// Production seeding (minimal data)
public static async Task SeedProductionData(IApplicationBuilder app)
{
    using var scope = app.ApplicationServices.CreateScope();
    var catalogContext =
scope.ServiceProvider.GetRequiredService<CatalogContext>();

    // Seed only essential data
    await SeedEssentialCategories(catalogContext.Types);
    await SeedEssentialBrands(catalogContext.Brands);
}
```

Q2: How do you ensure seed data doesn't conflict with existing data?

Answer: I implement idempotent seeding:

```csharp
// CatalogContextSeed.cs
public static class CatalogContextSeed
{
    public static void SeedData(IMongoCollection<Product> productCollection)
    {
        // Check if products already exist
        bool checkProducts = productCollection.Find(b => true).Any();

        if (!checkProducts)
        {
            var productsData = File.ReadAllText("Data/SeedData/products.json");
            var products =
JsonSerializer.Deserialize<List<Product>>(productsData);

            if (products != null)
            {
                foreach (var item in products)
                {
```

```
                    productCollection.InsertOneAsync(item);
                }
            }
        }
    }
}

// SQL Server seeding with transactions
public static async Task SeedAsync(OrderContext orderContext,
ILogger<OrderContextSeed> logger)
{
    using var transaction = await orderContext.Database.BeginTransactionAsync();

    try
    {
        if (!orderContext.Orders.Any())
        {
            orderContext.Orders.AddRange(GetPreconfiguredOrders());
            await orderContext.SaveChangesAsync();
            logger.LogInformation("Ordering Database seeded");
        }

        await transaction.CommitAsync();
    }
    catch (Exception ex)
    {
        await transaction.RollbackAsync();
        logger.LogError(ex, "Error seeding database");
        throw;
    }
}
```

Factory Pattern Interview Questions

Q1: How would you implement the Factory pattern for service client creation in your microservices?

Answer: I would implement a factory for gRPC service clients:

```csharp
```

```
// Service client interface
public interface IServiceClient
{
    Task<TResponse> CallAsync<TRequest, TResponse>(TRequest request) where
TRequest : class where TResponse : class;
}

// gRPC client implementation
public class GrpcServiceClient : IServiceClient
{
    private readonly Channel _channel;

    public GrpcServiceClient(string address)
    {
        _channel = new Channel(address, ChannelCredentials.Insecure);
    }

    public async Task<TResponse> CallAsync<TRequest, TResponse>(TRequest request)
        where TRequest : class
        where TResponse : class
    {
        var client = _channel.CreateGrpcClient<TResponse>();
        return await client.CallAsync(request);
    }
}

// Factory
public class ServiceClientFactory
{
    private readonly IConfiguration _configuration;

    public ServiceClientFactory(IConfiguration configuration)
    {
        _configuration = configuration;
    }

    public IServiceClient CreateClient(string serviceName)
    {
        var address = _configuration[$"Services:{serviceName}:Url"];
        return new GrpcServiceClient(address);
    }
}
```

```csharp
// Usage in Basket service
public class BasketService
{
    private readonly ServiceClientFactory _factory;

    public BasketService(ServiceClientFactory factory)
    {
        _factory = factory;
    }

    public async Task<decimal> GetDiscount(string productName)
    {
        var client = _factory.CreateClient("Discount");
        var request = new GetDiscountRequest { ProductName = productName };
        var response = await client.CallAsync<GetDiscountRequest,
CouponModel>(request);
        return response.Amount;
    }
}
```

Q2: What are the benefits of using Factory pattern in your microservices architecture?

Answer: The Factory pattern provides:

1. **Loose coupling** - Services don't depend on concrete client implementations
2. **Centralized configuration** - All service URLs in one place
3. **Easy testing** - Can mock factory for unit tests
4. **Flexibility** - Can switch between gRPC, HTTP, or other protocols
5. **Error handling** - Centralized error handling for service calls

Example of protocol switching:

```csharp
public class ServiceClientFactory
{
    public IServiceClient CreateClient(string serviceName, string protocol =
"grpc")
    {
        var address = _configuration[$"Services:{serviceName}:Url"];

        return protocol switch
```

```
        {
            "grpc" => new GrpcServiceClient(address),
            "http" => new HttpServiceClient(address),
            "amqp" => new AmqpServiceClient(address),
            _ => throw new ArgumentException($"Unknown protocol: {protocol}")
        };
    }
}
```

Abstract Factory Interview Questions

Q1: How would you implement Abstract Factory for database operations in your microservices?

Answer: I would implement an abstract factory for database families:

```csharp
// Abstract products
public interface IDbConnection
{
    Task OpenAsync();
    Task CloseAsync();
    Task ExecuteAsync(string sql);
}

public interface IDbCommand
{
    void SetCommand(string command);
    Task<object> ExecuteScalarAsync();
    Task<List<T>> QueryAsync<T>(string sql);
}

public interface IDbTransaction
{
    Task BeginAsync();
    Task CommitAsync();
    Task RollbackAsync();
}

// Abstract factory
```

```
public interface IDbFactory
{
    IDbConnection CreateConnection(string connectionString);
    IDbCommand CreateCommand();
    IDbTransaction CreateTransaction();
}

// SQL Server implementation
public class SqlConnection : IDbConnection
{
    private readonly SqlConnection _connection;

    public SqlConnection(string connectionString)
    {
        _connection = new SqlConnection(connectionString);
    }

    public async Task OpenAsync() => await _connection.OpenAsync();
    public async Task CloseAsync() => await _connection.CloseAsync();
    public async Task ExecuteAsync(string sql) => await new SqlCommand(sql,
_connection).ExecuteNonQueryAsync();
}

public class SqlDbFactory : IDbFactory
{
    public IDbConnection CreateConnection(string connectionString)
    {
        return new SqlConnection(connectionString);
    }

    public IDbCommand CreateCommand()
    {
        return new SqlCommand();
    }

    public IDbTransaction CreateTransaction()
    {
        return new SqlTransaction();
    }
}

// MongoDB implementation
public class MongoDbConnection : IDbConnection
```

```
{
    private readonly IMongoDatabase _database;

    public MongoDbConnection(string connectionString, string databaseName)
    {
        var client = new MongoClient(connectionString);
        _database = client.GetDatabase(databaseName);
    }

    public async Task OpenAsync() { /* MongoDB manages connections */ }
    public async Task CloseAsync() { /* MongoDB manages connections */ }
    public async Task ExecuteAsync(string json) { /* Execute MongoDB command */ }
}

public class MongoDbFactory : IDbFactory
{
    public IDbConnection CreateConnection(string connectionString)
    {
        return new MongoDbConnection(connectionString, "ecommerce");
    }

    public IDbCommand CreateCommand()
    {
        return new MongoDbCommand();
    }

    public IDbTransaction CreateTransaction()
    {
        return new MongoDbTransaction();
    }
}

// Usage in service
public class OrderService
{
    private readonly IDbFactory _dbFactory;

    public OrderService(IDbFactory dbFactory)
    {
        _dbFactory = dbFactory;
    }

    public async Task CreateOrder(Order order)
```

```
    {
        using var connection = _dbFactory.CreateConnection(_connectionString);
        using var transaction = _dbFactory.CreateTransaction();

        await connection.OpenAsync();
        await transaction.BeginAsync();

        try
        {
            var command = _dbFactory.CreateCommand();
            command.SetCommand($"INSERT INTO Orders VALUES ('{order.Id}',
'{order.UserName}', {order.TotalPrice})");
            await command.ExecuteScalarAsync();

            await transaction.CommitAsync();
        }
        catch
        {
            await transaction.RollbackAsync();
            throw;
        }
        finally
        {
            await connection.CloseAsync();
        }
    }
}
```

Q2: How do you choose between Factory and Abstract Factory patterns in your microservices?

Answer: I choose based on complexity:

Use Factory when:
- Creating single type of objects
- Simple object creation logic
- One implementation family

Example: Creating service clients
```csharp
// Simple factory
public class ServiceClientFactory
```

```
{
    public IServiceClient CreateClient(string serviceName)
    {
        return new GrpcServiceClient(GetServiceUrl(serviceName));
    }
}
```

Use Abstract Factory when:
- Creating families of related objects
- Multiple implementation families
- Objects must work together

Example: Database operations (Connection, Command, Transaction)
```csharp
// Abstract factory
public interface IDbFactory
{
    IDbConnection CreateConnection();
    IDbCommand CreateCommand();
    IDbTransaction CreateTransaction();
}
```

In my microservices, I would use:
- **Factory** - For service clients, HTTP clients, message producers
- **Abstract Factory** - For database families, message broker families, cache families

---

Final Summary

Key Architectural Decisions in Your Project

1. **Polyglot Persistence** - Redis for cache, MongoDB for catalog, SQL for orders
2. **Hybrid Communication** - REST for frontend, gRPC for inter-service
3. **Event-Driven** - RabbitMQ with MassTransit for async communication
4. **CQRS + MediatR** - Clean separation of read/write operations
5. **Observability** - Elasticsearch + Kibana for centralized logging
6. **API Versioning** - URL path versioning for backward compatibility

Best Practices Demonstrated

1. **Separation of Concerns** - Clean architecture layers
2. **Dependency Injection** - Loose coupling between components
3. **Idempotent Operations** - Safe retry mechanisms
4. **Configuration Management** - Environment-specific settings
5. **Container Orchestration** - Docker Compose for development

Areas for Enhancement

1. **Authentication/Authorization** - MSAL integration
2. **Resilience** - Polly for fault tolerance
3. **Monitoring** - Health checks and metrics
4. **Testing** - Comprehensive unit and integration tests
5. **Documentation** - API documentation and architecture diagrams

This interview guide covers all the key microservices concepts with practical examples from your actual implementation, making it an excellent resource for technical discussions and interviews.